

Diagrams Explained

TOPLAB EHR FRONTEND SYSTEM ARCHITECTURE

This document provides a detailed explanation for each of the diagrams and charts used to model the architecture, processes, and data flow of the Toplab EHR Frontend system.

1. Data Flow Diagrams (DFD)

Data Flow Diagrams map the flow of information for any process or system. For the EHR Frontend, this represents how user inputs travel through the frontend application to the mock API (and eventually the backend), and how data is returned to the view.

- **Stage 0 (Context Diagram):** Shows the EHR System as a single central process interacting with external entities such as the Medical Staff (Doctors/Nurses), Lab Technicians, and Admin. It highlights the main data inputs (e.g., Patient Details, Lab Results) and outputs (e.g., Reports, Encounters, Invoices).
- **Stage 1:** Breaks down the central process into sub-processes. For our system, this includes `Patient Management`, `Encounter/Queue Tracking`, `Laboratory Processing`, and `Billing & Reports`. It shows the data stores (e.g., Patient DB, Inventory DB, Encounter DB) these processes interact with.
- **Stage 2:** Provides a deeper dive into specific sub-processes. For instance, the Laboratory Processing process is broken down into *Request Test*, *Process Sample*, and *Release Result*, showing the exact states a Lab Result passes through.

2. Structured Chart

The Structured Chart provides a hierarchical visual representation of the system's module organization. For our Next.js frontend, this maps directly to our routing and component structure:

- **Root Level:** Represents the main `App` layout.
- **Level 1 Modules:** Distinguishes between Public Routes (Login) and Protected Routes (Dashboard, Patients, Lab Results, Services, etc.).
- **Level 2 Modules:** Shows the UI components nested within pages (e.g., `PatientsPage` relies on `PageHeader`, `SearchInput`, `StatusBadge`, and the data table).

This chart is vital for understanding the parent-child relationships and props passing between React components.

3. HIPO Diagrams (Hierarchical Input Process Output)

HIPO diagrams describe the system from a functional perspective. Each major function is broken down into its Inputs, Process, and Outputs.

Example Function: Add New Patient

INPUT	User fills out the Patient Form (Name, DOB, Sex, Contact, Branch).
PROCESS	The frontend validates the form data using Zod schemas, formats the dates, and triggers the <code>usePatients</code> mutation hook.
OUTPUT	The mock API receives the payload, a new patient record is generated, the UI state updates, and a success toast notification is displayed.

4. Entity-Relationship Diagram (ERD)

The ERD illustrates the relationships between the different data models defined in our `src/types` directory. This diagram ensures the development team understands the shape of the data they are querying and mutating.

- **Entities:** `Patient`, `Encounter`, `LabResult`, `Service`, `StaffMember`, `InventoryItem`, `Invoice`.
- **Relationships:**
 - A Patient can have *many* Encounters (1:N).
 - An Encounter can involve *many* LabResults and *many* Services (N:M or 1:N depending on structure).
 - An Invoice is tied to *one* Encounter and *one* Patient (1:1).

5. Data Dictionary

While not an image, the Data Dictionary complements the ERD by providing a strict definition for every field within the entities.

Entity	Field	Data Type	Definition / Constraints
Patient	<code>status</code>	Enum	Must be one of <code>['active', 'inactive', 'deceased']</code> . Default is 'active'.
Service	<code>price</code>	Integer	Numeric value representing Philippine Pesos (PHP). Must be ≥ 0 .